Day 151: Feature Engineering for Time Series: Lags, Rolling Statistics, Calendar Features, and Leakage

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 22, Day 5 | Complete study guide

Watch: Kishan Manani - Feature Engineering for Time Series Forecasting (PyData London 2022) | Cross-Validation for Time Series Forecasting, Python Tutorial (Egor Howell)

Script: `day151_feature_engineering.py` · every number in this guide is copied by program from `day151_run_output.txt`, the real output of that script.

Forecasting can be turned into ordinary regression: describe every month by features known before it, and fit a model from features to value. Today builds those features from scratch (checked against pandas and statsmodels to 1.8e-13 or better) and then breaks them on purpose. A rolling mean that includes the month itself gave a test error of 0.000; a centred trend looked 17 times better than a trend line; shuffled cross-validation promised 0.581 for a model whose real error was 1.978. Honestly built and chosen, the regression forecasts were close to Day 149's ARIMA on CO2 but no better, and the feature set chosen by validation ranked 4 of 6 later.

0. Where today fits

DEFINITION — start here, no prior knowledge assumed

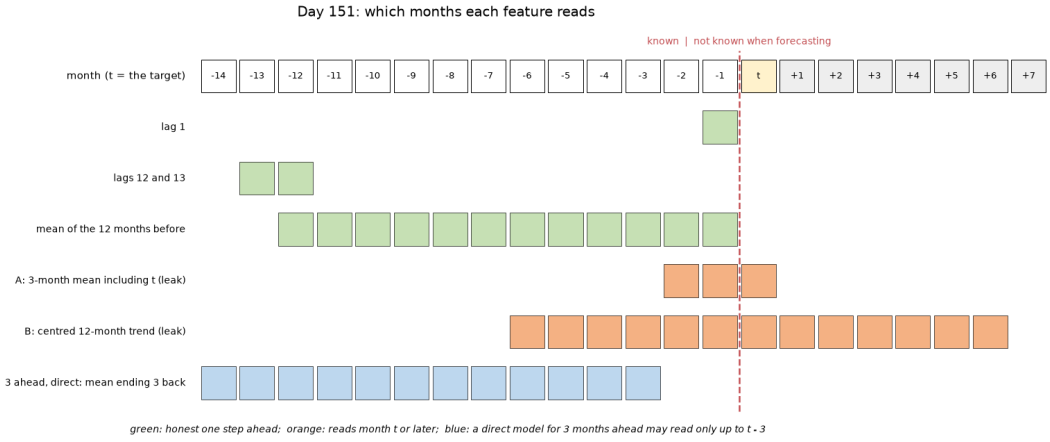
A **feature** is a number that describes a row. To forecast with regression, each month becomes a row whose target is that month's value and whose features are things known *before* it: the value 1 month earlier (**lag 1**), 12 months earlier (**lag 12**), the mean of the last 12 months (a **rolling statistic**), which calendar month it is (**calendar features**), and how much time has passed (a **trend**).

Leakage is any feature, or any step of the evaluation, that uses information which would not exist at the moment of forecasting. Nothing crashes when it happens: the error simply looks smaller than it will ever be in use.

REAL-WORLD SCENARIO

A shop predicts next week's sales from last week's, the same week last year, and whether a holiday falls in it. A model that accidentally reads 'this week's total' (filled in on Sunday night) will look perfect in testing and be useless on Monday morning, when the prediction is actually needed.

SYSTEM DIAGRAM



Which months each feature reads. Everything right of the red line is unknown when the forecast is made.

Today's part for the week's build

`day151_exercise.py` is today's parts (lag, rolling statistics, month and Fourier columns, the design matrix for any horizon, least squares, recursive and direct forecasts, nearest neighbours, and shuffled and time-ordered cross-validation), checked by hand: a lag, a rolling mean with and without the current month, the month columns, one Fourier pair, the lags a 3-month direct model reads, a straight line fitted exactly, and both forecast strategies on a series that halves every month.

ACTUAL OUTPUT

```
lag 1 of 1, 2, 3, 4: nan, 1, 2, 3; lag gives [nan, 1.0, 2.0, 3.0]
mean of the 2 months before, on 1..5: nan, nan, 1.5, 2.5, 3.5; rolling gives [nan, nan, 1.5, 2.5,
    3.5]
the same window including this month (shift 0, a leak): nan, 1.5, 2.5, 3.5, 4.5; gives [nan, 1.5,
    2.5, 3.5, 4.5]
month columns for January, February, December: all zeros, a 1 in column 1, a 1 in column 11;
    month_dummies gives 0 ones, column 1, column 11
one Fourier pair for April (month 3 from January): sin(2 pi 3/12) = 1, cos = 0; fourier gives
    [1.0, 0.0]
a 3-month direct model with lags 1 and 12 reads lags 3 and 14: ['intercept', 'lag 3', 'lag 14']
least squares on y = 2 + 3x, exactly: [2.0, 3.0]; cv_random 3.0e-16, cv_time 2.4e-15
a series that halves every month (64, ..., 1); with y = 0.5 x last month, the next three are 0.5,
    0.25, 0.125: recursive [0.5, 0.25, 0.125], direct [0.5, 0.25, 0.125]
2 nearest neighbours of 1.2 among 0, 1, 2, 10 are 1 and 2: knn gives [1.5]
```

Data: Day 147's two series. CO2 is trained up to 1999-12 and tested on 2000–2001, as on Days 147–150; the orders are used on a log scale. Every choice of features is made on earlier origins only; the test months are scored once.

How to read each section

Step	What it gives you
DEFINITION	What the idea is, in plain words. Start here.
REAL-WORLD SCENARIO	An everyday situation where the same idea shows up.
MATHEMATICS	The formula, when the section introduces one.
SPECIFICATION	The exact setup of the experiment.
IMPLEMENTATION	The real code from the script.
ACTUAL OUTPUT	What the script really printed, copied unchanged.
What it means	How to read the numbers, and what they do and do not show.
Why choose this? / Why NOT this?	When the idea is the right tool, and what it costs.

1. Building the features

DEFINITION — start here, no prior knowledge assumed

Each feature is a column, one row per month. A lag shifts the series down; a rolling statistic summarises a window that **ends before** the target; the month columns are 11 zeros-and-ones (January is the baseline, held by the constant); Fourier columns describe the season with a few smooth sine and cosine waves instead. Rows whose features reach back before the first month are incomplete and are left out of the fit.

The argument h makes the same function serve a model that forecasts h months ahead: every lag moves h – 1 further back and every window ends h months before the target, so no feature reads a month the forecaster could not have seen.

REAL-WORLD SCENARIO

A weather forecaster's notebook: yesterday's temperature, the same date last year, the average of the past week, and the season. All written down before the day being forecast.

MATHEMATICS

lag k: $x_t = y_{t-k}$; *rolling mean:* $x_t = (y_{t-s-w+1} + \ldots + y_{t-s}) / w$ $(s = shift \geq 1)$

Fourier pair j: $\sin(2\pi\, j\, m / 12), \cos(2\pi\, j\, m / 12)$, m = month 0 (January) to 11

for a model h months ahead: lag $k \rightarrow$ lag $k + h - 1$, shift $s = h$

IMPLEMENTATION

```
def lag(y, k):
    out = np.full(len(y), np.nan)
    out[k:] = y[:len(y) - k]
    return out
def rolling(y, w, stat, shift=1):
    out = np.full(len(y), np.nan)
    win = np.lib.stride_tricks.sliding_window_view(y, w)
    first = w - 1 + shift
    out[first:] = stat(win, axis=1)[:len(y) - first]
    return out
def month_dummies(months):
    mo = months.astype("datetime64[M]").astype(int) % M
    return (mo[:, None] == np.arange(1, M)).astype(float)
def fourier(months, K):
    mo = months.astype("datetime64[M]").astype(int) % M
    k = np.arange(1, K + 1)
    return np.column_stack([np.sin(2 * np.pi * mo[:, None] * k / M), np.cos(2 * np.pi * mo[:,
        None] * k / M)])
```

IMPLEMENTATION

```
def make_features(y, start, spec, h=1):
    months = np.datetime64(start, "M") + np.arange(len(y))
    cols, names = [np.ones(len(y))], ["intercept"]
    for k in spec.get("lags", ()):
        cols.append(lag(y, k + h - 1)); names.append(f"lag {k + h - 1}")
    for w, stat in spec.get("rolling", ()):
        cols.append(rolling(y, w, np.mean if stat == "mean" else np.std, shift=h));
            names.append(f"{stat} of {w}, ending {h} back")
    if spec.get("trend"):
        cols.append(np.arange(len(y)) / 120.0); names.append("trend")
    cal = spec.get("calendar")
    if cal == "months":
        D = month_dummies(months); cols += list(D.T); names += [f"month {j + 1}" for j in range(1,
            M)]
    elif cal:
        D = fourier(months, cal); cols += list(D.T); names += [f"wave {j}" for j in
            range(D.shape[1])]
    return np.column_stack(cols), names
```

ACTUAL OUTPUT

```
lag 12 against shift(12): largest difference 0.0e+00
mean of the 12 months before against rolling(12).mean().shift(1): 1.1e-13
sd of the 12 months before against rolling(12).std(ddof=0).shift(1): 2.4e-12
month columns against get_dummies: 0.0e+00 (11 columns; January is all zeros)
least squares on lags 1-13 against AutoReg(13): largest coefficient difference 1.8e-13
the first rows: 13 months have a missing lag and are left out of the fit (14 columns: intercept, lag 1, ..., lag 13)
Actual output from this lesson's run.
```

What it means

Our lags and month columns equal pandas' exactly (0.0e+00); the rolling mean and standard deviation agree to 1.1e-13 and 2.4e-12 with pandas' rolling(12) followed by shift(1). Least squares on lags 1–13 gives statsmodels' AutoReg coefficients to 1.8e-13: regression on lags *is* an autoregressive model.

WHY CHOOSE THIS?

Any regression method can now forecast, and features can be mixed freely: lags, windows, calendar, outside data.

WHY NOT THIS? (TRADEOFFS)

Every feature must be checked for what it reads; the model itself cannot tell an honest feature from a leaked one.

2. Four leaks

DEFINITION — start here, no prior knowledge assumed

Four ways the future gets in, each measured on the 2000–2001 test months with an honest model beside it.

A: a rolling mean computed with the current month included (the shift(1) forgotten). **B**: Day 147's centred trend used as a feature; it averages 6 months either side. **C**: a 24-month forecast scored with the true lags of every test month, i.e. one step ahead. **D**: features scaled with the mean and standard deviation of all months, test included.

REAL-WORLD SCENARIO

An exam where the answer sheet is stapled to the back: A copies it, B copies half of it, C marks each question after showing the previous answer, D only reads the class average.

MATHEMATICS

leak A: $mean_3(t) = (y_t + y_{t-1} + y_{t-2}) / 3 \;\Rightarrow\; y_t = 3\, mean_3(t) - y_{t-1} - y_{t-2}$

leak D: *with a constant, least squares on* $(x - \mu) / \sigma$ *gives the same fitted values for any* μ, σ

IMPLEMENTATION

```
def fit_ols(X, y):
    ok = ~np.isnan(X).any(axis=1) & ~np.isnan(y)
    return np.linalg.lstsq(X[ok], y[ok], rcond=None)[0]
```

IMPLEMENTATION

```
b = score("A: + 3-month mean that includes this month", np.c_[X, rolling(yc, 3, np.mean,
    shift=0)])
T = d147.centred_ma(yc); has = ~np.isnan(T)
score("honest: lags, on the 18 months B can reach", X, has)
score("B: lags + centred 12-month trend (Day 147)", np.c_[X, T], has)
C, _ = make_features(yc, tc[0], dict(calendar="months"))
score("honest: straight trend line + months", np.c_[C, np.arange(len(yc)) / 120.0], has)
score("B: centred 12-month trend + months", np.c_[C, T], has)
```

ACTUAL OUTPUT

```
                                features  train MAE  test MAE  months
                honest: lags 1, 2, 12, 13      0.308     0.292      24
   A: + 3-month mean that includes this month      0.000     0.000      24
   honest: lags, on the 18 months B can reach      0.308     0.340      18
   B: lags + centred 12-month trend (Day 147)      0.293     0.341      18
         honest: straight trend line + months      1.512     3.056      18
           B: centred 12-month trend + months      0.218     0.179      18
B reads 6 months past the target, so it does not exist for the last 6 test months; added to the lags it changes little
    (0.340 to 0.341), but in place of a trend line it looks 17 times better
A's coefficients on lag 1, lag 2 and the 3-month mean: -1.000, -1.000, 3.000 -- 3 x mean - lag 1 - lag 2 is this month
    itself
C: the honest model's 2000-2001 error one step ahead (true lags every month) 0.292; forecast from 1999-12 for 24 months
    0.664 -- a 24-month forecast must not be scored with lags it could not have known
D: scaling the features with all months (test included) instead of the training months: least-squares forecasts change by at
    most 4.0e-13
   (least squares with a constant gives the same fit for any shift and scaling of the features, so this leak cannot matter
       here)
Actual output from this lesson's run.
```

What it means

A is total: with lags 1 and 2 already present, least squares found coefficients -1.000, -1.000 and 3.000, which rebuild the month itself. Training and test error are both 0.000. The honest model scores 0.292.

B depends on the company it keeps. Next to the lags it changes almost nothing (0.340 to 0.341); in place of a straight trend line it turns 3.056 into 0.179, 17 times better. A leak that does not show is still a leak: the centred trend does not exist for the last 6 months, which are exactly the months a forecast needs.

C: the same honest model scores 0.292 one step ahead but 0.664 when it really forecasts 24 months from 1999-12. **D** changed the forecasts by at most 4.0e-13: least squares with a constant is unaffected by shifting and scaling. Not every leak matters, and only measuring tells which.

WHY CHOOSE THIS?

Measuring each leak beside an honest model shows which are dangerous here (A, B, C) and which are harmless (D).

WHY NOT THIS? (TRADEOFFS)

D is harmless only for least squares; a method that depends on scale (nearest neighbours, regularised regression) can be affected.

3. Cross-validation: shuffled against time-ordered folds

DEFINITION — start here, no prior knowledge assumed

Cross-validation estimates a model's error on unseen rows by holding out part of the training data. Shuffled K-fold holds out a random fifth of the months at a time; time-ordered validation trains on everything before a block and tests on the block, as a real forecast would.

Two models on the same lag features: least squares, and **5-nearest neighbours** (5-NN), which predicts the mean target of the 5 training rows whose features are closest. Then the same comparison on the monthly *changes* instead of the levels.

REAL-WORLD SCENARIO

Judging a student by questions drawn at random from this term's homework, when the real exam is next term's material: they may only have memorised the neighbouring answers.

IMPLEMENTATION

```
def knn_fit_predict(Xtr, ytr, Xte, k=5):
    mu, sd = Xtr.mean(axis=0), Xtr.std(axis=0) + 1e-12
    A, B = (Xtr - mu) / sd, (Xte - mu) / sd
    d = ((B[:, None, :] - A[None, :, :]) ** 2).sum(axis=2)
    return ytr[np.argsort(d, axis=1)[:, :k]].mean(axis=1)
def cv_random(X, y, fit_predict, folds=5, seed=0):
    idx = np.random.RandomState(seed).permutation(len(y)); err = []
    for f in range(folds):
        te = idx[f::folds]; tr = np.setdiff1d(idx, te)
        err.append(np.abs(fit_predict(X[tr], y[tr], X[te]) - y[te]))
    return np.mean(np.concatenate(err))
```

IMPLEMENTATION

```
def cv_time(X, y, fit_predict, folds=5):
    edges = np.linspace(len(y) // 2, len(y), folds + 1).astype(int); err = []
    for a, b in zip(edges[:-1], edges[1:]):
        err.append(np.abs(fit_predict(X[:a], y[:a], X[a:b]) - y[a:b]))
    return np.mean(np.concatenate(err))
```

ACTUAL OUTPUT

```
            target, lags, model  shuffled CV  time CV  test (2000-01)
     CO2 level, 3, least squares        0.597    0.628           0.548
              CO2 level, 3, 5-NN        1.048    3.547           1.556
    CO2 level, 12, least squares        0.305    0.331           0.279
             CO2 level, 12, 5-NN        0.581    3.858           1.978
    CO2 level, 24, least squares        0.268    0.284           0.245
             CO2 level, 24, 5-NN        0.583    4.178           2.164
monthly change, 12, least squares        0.284    0.296           0.233
        monthly change, 12, 5-NN        0.243    0.262           0.280
5-NN on levels: shuffled CV promises 0.581, the test gives 1.978 (3.4 times worse); time-ordered CV warned (3.858)
why: in the shuffled folds, a held-out month's nearest training month (in features) is a median 1 month(s) away in time, and
    within 2 months for 51% of them
each time-ordered block is about 49 months long, against the test's 24, so it reaches further past its training rows: a
    likely reason it is more pessimistic than the test (not tested here)
for least squares, and for 5-NN on the monthly changes, the shuffled estimate stays close to the test
Actual output from this lesson's run.
```

What it means

For least squares the two estimates agree and both are close to the test (12 lags: 0.305 shuffled, 0.331 time-ordered, 0.279 test). This matches a published result: for purely autoregressive models shuffled cross-validation can be valid (Bergmeir, Hyndman and Koo, 2018).

For 5-NN on the levels it is badly wrong: shuffled folds promise 0.581, the test gives 1.978, 3.4 times worse. A shuffled fold tests a month whose neighbours in time are in the training rows, and their features are almost identical. Measured: a held-out month's nearest training month was a median of 1 month away in time, and within 2 months for 51% of them. On a rising series the test months are above anything in training, and nearest neighbours cannot extrapolate. On the monthly changes, which do not rise, 5-NN's shuffled estimate (0.243) is close to the test (0.280).

The time-ordered estimate for 5-NN (3.858) was more pessimistic than the test. Its blocks are about 49 months long against the test's 24, so they reach further past the training rows: a likely reason, not tested here.

WHY CHOOSE THIS?

Time-ordered validation asks the same question as the forecast, so it warns where shuffled folds flatter.

WHY NOT THIS? (TRADEOFFS)

It uses fewer, later rows per estimate; its blocks must be sized like the real forecast or it may be pessimistic.

4. Choosing the features for CO2; recursive and direct forecasts

DEFINITION — start here, no prior knowledge assumed

Six candidate feature sets, from lags alone to lags with a trend, the month columns (or 2 Fourier pairs) and the 12-month mean and standard deviation. Each is scored by forecasting 24 months from 157 origins between 1975 and 1988, and the lowest mean error is the choice; the test months are not touched.

A model trained one step ahead forecasts further by **recursion**: forecast next month, treat it as data, repeat. The alternative is **direct**: a separate model for each horizon h, trained only on features at least h months old.

REAL-WORLD SCENARIO

Recursive: plan a road trip one town at a time, each leg starting where the last plan ended. Direct: plan the distance to each town separately from home.

MATHEMATICS

$$\textit{recursive:}\quad \hat{y}_{T+1} = f(y_T, y_{T-1}, \ldots),\quad \hat{y}_{T+2} = f(\hat{y}_{T+1}, y_T, \ldots),\quad \ldots$$

$$\textit{direct:}\quad \hat{y}_{T+h} = f_h(y_T, y_{T-1}, \ldots),\quad \text{one } f_h \text{ per horizon}$$

IMPLEMENTATION

```
def recursive_forecast(y, start, spec, b, h):
    yy = np.r_[y, np.full(h, np.nan)]
    for i in range(len(y), len(y) + h):
        X, _ = make_features(yy[:i + 1], start, spec)
        yy[i] = X[-1] @ b
    return yy[len(y):]
def direct_forecast(y, start, spec, h):
    out = np.empty(h)
    for j in range(1, h + 1):
        X, _ = make_features(y, start, spec, h=j); b = fit_ols(X, y)
        Xf, _ = make_features(np.r_[y, np.full(j, np.nan)], start, spec, h=j)
        out[j - 1] = Xf[-1] @ b
```

ACTUAL OUTPUT

```
validation: 157 origins, 1975-01 to 1988-01, each forecasting 24 months (forecasts end by 1989-12)
                                          features  validation  test 2000-01
                                         lags 1-13       0.473         0.706
                                    lags 1, 12, 13       0.496         0.670
                                    trend + months       2.269         3.062
                     lags 1, 12, 13 + trend + months       0.589         0.396
                    lags 1, 12, 13 + trend + 2 waves       0.598         0.367
 lags 1, 12, 13 + trend + months + 12-month mean, sd       0.517         0.352
validation chooses: lags 1-13
2000-01 to 2001-12, MAE (ppm): lags 1-13, recursive 0.706, direct 0.727; airline ARIMA (Day 149) 0.412
Actual output from this lesson's run.
```

What it means

Validation chose **lags 1–13** (0.473). On the test it forecast with a mean error of 0.706 ppm (direct: 0.727), worse than Day 149's airline ARIMA (0.412). Three other sets did much better on the test (0.352, 0.367, 0.396), but picking one now would be choosing on the test, so the result stands as validation chose it.

WHY CHOOSE THIS?

Recursive needs one model; direct never feeds its own errors back in, and each horizon gets its own coefficients.

WHY NOT THIS? (TRADEOFFS)

Recursive compounds its errors; direct needs h models and loses the most recent months as h grows.

5. Rolling origins, and whether the choice held

DEFINITION — start here, no prior knowledge assumed

The same 97 origins as Days 149 and 150 (1990–1998), each forecasting 24 months, for the chosen features (recursive and direct), the airline ARIMA and same month last year. Then every candidate set on the same origins, to see whether the validation ranking from 1975–1988 held a decade later.

REAL-WORLD SCENARIO

A recipe chosen because it won last year's contest: worth checking whether it still wins with this year's ingredients.

ACTUAL OUTPUT

```
97 origins (7.0 s); features: lags 1-13
     MAE (ppm), months ahead       1       6      12      24
       regression, recursive   0.300   0.588   0.701   1.058
          regression, direct   0.300   0.524   0.747   1.188
          airline ARIMA, CSS   0.264   0.451   0.715   1.099
        same month last year   1.362   1.428   1.560   3.137
recursive regression beats the airline model at 5 of 24 horizons; direct beats recursive at 8 of 24
did the validation ranking (1975-1988 origins) hold in 1990-1998? mean error over 24 months, recursive:
                                          lags 1-13  validation 0.473  1990-1998 0.784
                                    lags 1, 12, 13  validation 0.496  1990-1998 0.786
   lags 1, 12, 13 + trend + months + 12-month mean, sd  validation 0.517  1990-1998 0.677
                       lags 1, 12, 13 + trend + months  validation 0.589  1990-1998 0.725
                    lags 1, 12, 13 + trend + 2 waves  validation 0.598  1990-1998 0.724
                                    trend + months  validation 2.269  1990-1998 2.415
validation's choice ranked 4 of 6 in 1990-1998; the best there: lags 1, 12, 13 + trend + months + 12-month mean, sd
Actual output from this lesson's run.
```

What it means

The regression and the airline model are close: 0.300 against 0.264 one month ahead, 1.058 against 1.099 at 24; the regression is better at only 5 of 24 horizons, and direct beat recursive at 8 of 24.

The validation ranking did not hold. The chosen set's error rose from 0.473 to 0.784; the set with the trend, months and 12-month statistics did best in the 1990s (0.677), and validation's choice ranked 4 of 6. Why the ranking changed was not tested here.

WHY CHOOSE THIS?

Rolling origins show the whole error curve, and whether a choice survives a change of period.

WHY NOT THIS? (TRADEOFFS)

A choice made on one period carries no guarantee for the next, however honestly it was made.

6. Electrical-equipment orders

DEFINITION — start here, no prior knowledge assumed

The same steps on the log orders: the six sets validated on 25 origins in 1998–1999 (12 months ahead), then 73 rolling origins 2001–2007 and the crash.

REAL-WORLD SCENARIO

A demand model tuned on one stretch of years, then used on the next: the test is whether its features still describe the series.

ACTUAL OUTPUT

```
validation (25 origins, 12 months ahead, log scale) chooses: lags 1, 12, 13 + trend + months + 12-month mean, sd (0.0527;
    lags 1-13 0.0771, lags 1, 12, 13 0.0602, trend + months 0.0548, lags 1, 12, 13 + trend + months 0.0534, lags 1, 12, 13 +
    trend + 2 waves 0.0641)
rolling origins 2001-01 to 2007-01 (73), mean absolute % error by months ahead:
                              1       6      12
   regression, recursive   2.38%   6.26%  12.05%
      regression, direct   2.38%   6.76%  18.28%
      airline ARIMA, CSS   1.92%   4.39%   6.26%
    same month last year   5.31%   5.13%   4.83%
did the validation ranking (1998-1999 origins) hold in 2001-2007? mean error over 12 months, recursive:
   lags 1, 12, 13 + trend + months + 12-month mean, sd  validation 5.27 (log error x 100)  2001-2007 6.92%
                        lags 1, 12, 13 + trend + months  validation 5.34 (log error x 100)  2001-2007 7.80%
                                       trend + months  validation 5.48 (log error x 100)  2001-2007 12.26%
                                       lags 1, 12, 13  validation 6.02 (log error x 100)  2001-2007 5.09%
                      lags 1, 12, 13 + trend + 2 waves  validation 6.41 (log error x 100)  2001-2007 6.19%
                                             lags 1-13  validation 7.71 (log error x 100)  2001-2007 5.59%
validation's choice ranked 4 of 6 in 2001-2007; the best there: lags 1, 12, 13; validation's top three (all with a trend
    line: True) ranked 4, 5, 6
the crash (train up to 2007-12, 24 months): regression 21.83%, airline ARIMA 23.86%, same month last year 18.04%
Actual output from this lesson's run.
```

What it means

Validation chose the set with a trend line, months and 12-month statistics. In 2001–2007 it lost to the airline model at every horizon shown (2.38% against 1.92% one month ahead, 12.05% against 6.26% at 12), and same month last year (4.83%) beat it a year ahead; direct was worse still at 12 months (18.28%).

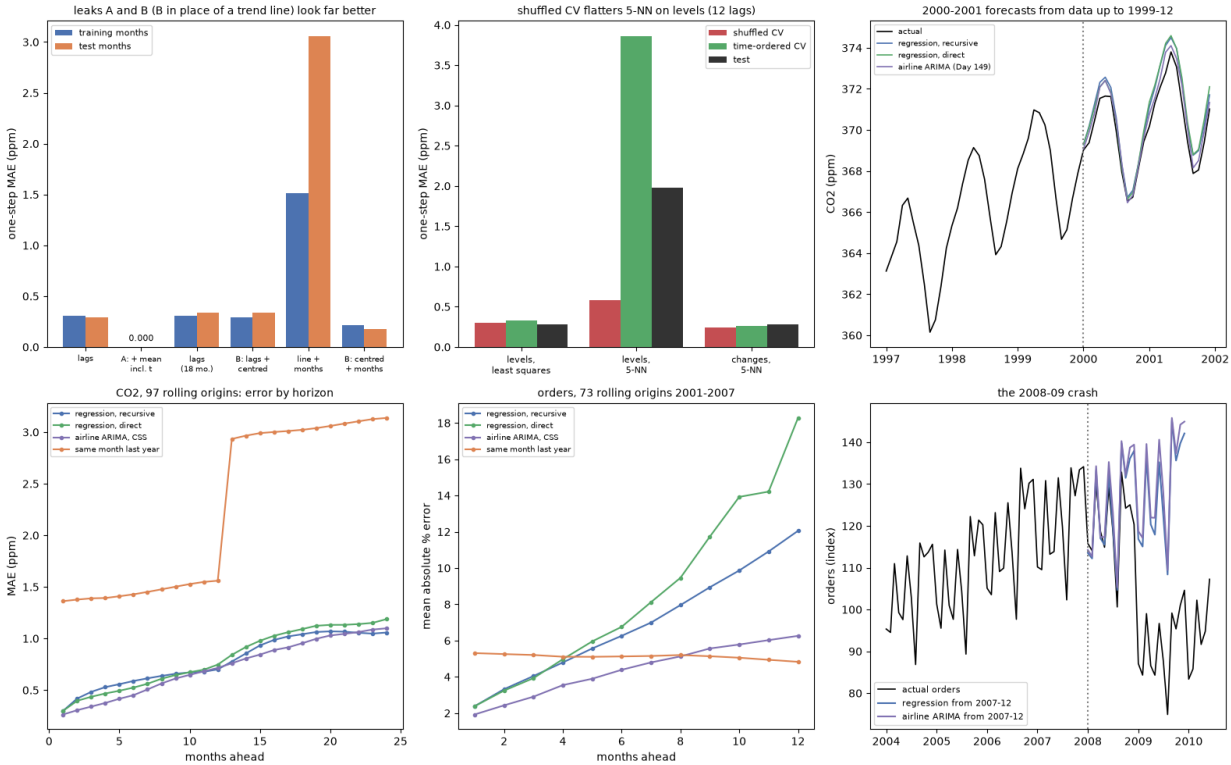
Validation's top three all contained a trend line, and in 2001–2007 they ranked 4, 5, 6; plain lags 1, 12, 13 did best (5.09% against the choice's 6.92%). A likely reason, not tested here: a straight trend fitted on 1996–1999 need not describe the years after 2001. In the crash the regression (21.83%) beat the airline model (23.86%) but not same month last year (18.04%).

WHY CHOOSE THIS?

Regression features are easy to add, and validation keeps the choice honest.

WHY NOT THIS? (TRADEOFFS)

With a short history, validation covers one kind of period; a trend feature learned there can mislead badly later.



Top: training and test error with and without the leaks; shuffled and time-ordered cross-validation against the test; the 2000–2001 forecasts. Bottom: CO2 rolling-origin error by horizon; the orders' rolling-origin error; the crash.

Interview preparation — technical questions

Q: How do you turn forecasting into a regression problem?

A: Make one row per time step with the value as target and features known before it: lags, windows ending before the target, calendar columns, trend.

Q: Name a common leak in rolling features.

A: Computing the window with the current value inside it; here a 3-month mean did this and the test error became 0.000.

Q: Can you use K-fold cross-validation on time series?

A: For least squares on lags it agreed with the test here; for 5-NN on a trending series it promised 0.581 against a real 1.978. Use time-ordered validation unless you have checked.

Q: Recursive or direct multi-step forecasts?

A: Measure both: on CO2 direct won at 8 of 24 horizons, on the orders it was worse a year ahead.

Q: Does scaling with all the data leak?

A: In principle yes; for least squares with a constant it changed forecasts by 4.0e-13, so it cannot matter there; for scale-dependent methods it can.

Interview preparation — theoretical questions

Q: What is data leakage?

A: Using information at training or evaluation time that would not exist at the moment of the real prediction.

Q: Why is a one-step score not a 24-month score?

A: One step ahead, every lag is a real value; 24 months ahead most of them are the model's own forecasts. Here: 0.292 against 0.664.

Q: Why did nearest neighbours fail on the levels?

A: They can only average training targets, and the test months lie above every training value of a rising series.

Q: What are Fourier features?

A: Sine and cosine waves with periods of 12, 6, ... months: a smooth description of the season with fewer columns than 11 month columns.

Key takeaway and what's next

KEY TAKEAWAY

Features turn forecasting into regression: lags, windows that end before the target, calendar columns and a trend, each checked here against pandas and statsmodels. The danger is leakage, and it rarely announces itself: a forgotten shift gave a perfect score, a centred trend looked 17 times better, a one-step score made a 24-month error look less than half its size, and shuffled folds flattered a nearest-neighbour model 3.4 times. One leak measured as harmless. Built and chosen honestly, regression on lags was close to ARIMA on CO2 but not better, and the feature set chosen by validation did not stay the best in a later period. Tomorrow tries a recurrent network on the same windows.

- Features equal pandas' and AutoReg's to 1.8e-13 or better.
- Leak A gave 0.000; leak B 17 times better than a trend line; leak C 0.292 against 0.664; leak D 4.0e-13.
- Shuffled CV for 5-NN on levels: 0.581 promised, 1.978 real.
- CO2 2000–2001: chosen regression 0.706, ARIMA 0.412; rolling origins nearly tied.
- Validation's choice later ranked 4 of 6 on both series.

Suggested watch order, if starting from zero

- 1. Kishan Manani - Feature Engineering for Time Series Forecasting (PyData London 2022) — lags, window and calendar features for forecasting with regression models, and the pitfalls.
- 2. Cross-Validation for Time Series Forecasting, Python Tutorial (Egor Howell) — time-ordered cross-validation for forecasts, to compare with today's shuffled-against-time experiment.

Day 152 preview

- **Recurrent networks for forecasting.** Week 10's LSTM parts, trained on windows of past months, are compared on the same origins with Day 149's ARIMA and today's regression.